Universidade Federal de Viçosa – Campus Florestal
Projeto Integrador 2025 - Arquitetura de Software
Equipe 2

Testes na Arquitetura

Pingo: Uma Aventura na Rede

O presente documento descreve a estratégia de testes e a organização da arquitetura para o jogo “Pingo: Uma Aventura na Rede”, idealizado e desenvolvido pela Equipe 2 no âmbito do Projeto Integrador 2025.

Versão: 1.0

Data: 19/09/2025

Autor(a): Ana Carolina Fernandes, Arquiteta de Software

Sumário

1. Testes na Arquitetura..... 3
1.1. Testes Unitários.....3
1.2. Testes de Integração.....4
2. Organização das pastas.....5
2.1. Back-end.....5
3. Conclusão.....6

1. Testes na Arquitetura

A integração dos testes deve validar cada camada de maneira independente, com testes unitários, e também a integração entre elas, com testes de integração. É importante especificar como cada camada definida na arquitetura seguida pelo projeto (Arquitetura em camadas), as camadas devem ser testadas conforme descrito ao longo dessa seção.

1.1. Testes Unitários

Os **Testes Unitários** são utilizados para verificar pequenas partes de um código, como uma única classe, sendo mais simples e fáceis de implementar.

Os testes unitários devem chamar métodos de uma classe e verificar se eles retornam os resultados esperados. Desta maneira, é recomendado que sejam utilizados na camada ***Service***, para validar regras de negócio, não sendo necessário interagir com o banco de dados ou com as camadas web.

É possível utilizar este tipo de teste para a camada ***Model***, porém essa camada pode ser testada indiretamente por meio dos testes de outras camadas que se relacionam com ela, como ***Dao, Service*** e ***Controller***.

Abaixo está um exemplo de um teste unitário baseado na prova de conceito, utilizando um *mock* da *FaseDAO*.

```
Java
/**
 * Classe de teste para o FaseService.
 * Utiliza JUnit 5 e Mockito para criar testes unitários.
 */
@ExtendWith(MockitoExtension.class)
class FaseServiceTest {

    /**
     * Cria um mock do FaseDAO.
     * Um mock é um objeto falso que simula o comportamento de um objeto
real.
     */
    @Mock
    private FaseDAO faseDAO;

    /**
     * Cria uma instância do FaseService e injeta o mock do FaseDAO nele.
     */
    @InjectMocks
    private FaseService faseService;

    /**
     * Método de teste para o método listarFases do FaseService.
```

```
     */
    @Test
    void testListarFases() {
        /**
         * Define o que o mock do DAO deve retornar quando o método for
chamado
         */
        List<Fase> fasesMock = List.of(
                new Fase(3L, "Definindo o Caminho", false),
                new Fase(4L, "Navegando na Fibra Óptica", false)
        );
        when(faseDAO.listarTodas()).thenReturn(fasesMock);

        /**
         * Chama o método do serviço que está sendo testado
         */
        List<Fase> resultado = faseService.listarFases();

        /**
         * Faz verificações para garantir que o resultado é o esperado
         *
         * assertThat: Verifica se o resultado não é nulo, se o tamanho da
lista é 2 e se o nome da primeira fase é "Definindo o Caminho".
         */
        assertThat(resultado).isNotNull();
        assertThat(resultado.size()).isEqualTo(2);
        assertThat(resultado.get(0).getNome()).isEqualTo("Definindo o
Caminho");

        /**
         * verify: Verifica se o método listarTodas do DAO foi chamado
exatamente uma vez.
         */
        verify(faseDAO, times(1)).listarTodas();
    }
}
```

1.2. Testes de Integração

Os **Testes de Integração** envolvem mais classes, dependências e sistemas reais, como bancos de dados e serviços remotos. Seu objetivo é validar a comunicação entre diferentes partes da arquitetura. Além disso, quando se implementa testes de integração não há a necessidade de *mocks* de componentes internos.

Sua utilização deverá ser nas camadas ***Controller***, ***Service*** e ***Dao***. No ***Controller*** para simular requisições HTTP, utilizando um ***Service*** real ao invés de

um *mock*, e utilizando a camada ***Dao*** para testar operações em um banco de dados local.

Abaixo está um exemplo de um teste de integração envolvendo as camadas ***Controller***, que por sua vez invoca o ***Service***. O serviço então executa suas regras e chama o ***Dao***, que realiza a consulta real contra o banco de dados de teste.

```
Java
/**
 * @SpringBootTest carrega o contexto completo da aplicação, incluindo
services, DAOs e o banco de dados.
 * @AutoConfigureMockMvc adiciona a infraestrutura do MockMvc para realizar
chamadas HTTP.
 */
@SpringBootTest
@AutoConfigureMockMvc
class FaseControllerIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    @Sql("/test-fases.sql")
    void testGetFases() throws Exception {

        mockMvc.perform(get("/api/fases"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$", hasSize(2)))
                .andExpect(jsonPath("$[0].nome", is("Definindo o Caminho")))
                .andExpect(jsonPath("$[1].nome", is("Navegando na Fibra
Óptica")));
    }
}
```

2. Organização das pastas

2.1. Back-end

Para manter a organização, a estrutura de pastas dos testes deve ser um espelho da estrutura do código-fonte principal, como mostrado abaixo, com a pasta *test* sendo criada dentro da pasta *src*.

```shell
test.java.br.com.pingo/
├── controller/
│   └── UserControllerTest.java
├── dao/
│   └── UserDAOTest.java
└── service/
    └── UserServiceTest.java
```

3. Conclusão

A arquitetura do nosso sistema foi planejada em camadas para ser organizada e fácil de manter. Esta estratégia de testes é a nossa forma de garantir que esse plano funcione na prática. Com os testes, nós verificamos cada camada individualmente e, mais importante, garantimos que a comunicação entre elas seja perfeita.